DEPARTMENT OF COMPUTER APPLICATIONS

MASTER OF COMPUTER APPLICATIONS (MCA)

Final Year Project — Semester III (Academic Year 2025 - 2026)

ZeroTrustCloud: Implementing and Automating Identity-Centric Zero-Trust Architecture for Cloud-Native Environments

RESEARCH METHODOLOGY, DATA COLLECTION & EXPERIMENTAL PLAN

Domain: Cybersecurity & Supply Chain Defense

Proposed Platform: TrustMesh (Identity-Centric Zero-Trust Control Plane)

Project Period: September 4, 2026 – November 27, 2026

Current Project Status: **Research & Methodology / Experimental Planning Phase (Week 4)**

SUBMITTED BY

Hithaishi S P

GitHub: @HithaishiSP2004 | MCA Final Year, Semester III

Project Team Collaborators:

Baire Gowda (@bairegowda1003) • Sanjay (@sanju722002)

1. Executive Summary & Purpose of This Document

The ZeroTrustCloud project has concluded its preliminary three-week literature exploration phase, during which 20 research papers and technical publications across cloud-native security, SPIFFE/SPIRE workload attestation, Open Policy Agent (OPA) governance, Istio mutual TLS (mTLS), and software supply chain protection were systematically surveyed. The project is currently situated in Week 4: Research Methodology, Threat Modeling, and Experimental Data Planning.

The purpose of this document is to establish a rigorous scientific blueprint for conducting the remainder of the project. Specifically, this document:

- **Defines the research and engineering methodologies** connecting theoretical Zero-Trust principles (NIST SP 800-207) to practical multi-cloud Kubernetes deployments.
- **Formalizes the threat model** identifying high-priority cloud-native attack vectors that will be evaluated experimentally.
- **Outlines the empirical data collection strategy** specifying the exact schema, parameters, and telemetry metrics that will be generated during controlled security experiments.
- **Designs controlled experimental test scenarios** comparing a baseline cloud-native deployment against the hardened TrustMesh architecture.
- **Establishes the GitHub repository blueprint** clearly differentiating current research assets from planned implementation modules without premature fabrication.

Important Note on Project State: This document represents the operational methodology and experimental design. In adherence to academic integrity, no implementation code, live cluster deployments, or synthetic test measurements are claimed as completed; those activities are scheduled for the implementation and benchmarking phases (Weeks 5–10).

2. Research Methodology

The research follows a structured six-stage engineering lifecycle designed to ensure measurable, reproducible outcomes:

Phase 1: Literature Survey & Gap Extraction [Completed]

Systematically review 20 research papers and technical publications to understand state-of-the-art mechanisms in workload identity, Policy-as-Code, service mesh encryption, and supply chain security. Identify documented architectural blind spots (e.g., binary certificate trust, network policy misconfigurations, and untrusted control planes).

Phase 2: Threat Modeling & Security Scope Definition [Current — Week 4]

Conduct structured threat modeling across containerized application layers (identity, networking, cluster admission, supply chain, and runtime) to prioritize experimental attack vectors.

Phase 3: Architecture Design & Control Plane Formalization [Current — Week 4]

Formalize the TrustMesh architecture blueprint, defining component interaction protocols, policy evaluation interfaces, and data collection points.

Phase 4: Testbed Infrastructure & Prototype Implementation [Planned — Weeks 5–7]

Deploy baseline and Zero-Trust environments across Kubernetes clusters (AWS EKS / Azure AKS), integrating SPIRE, OPA Gatekeeper, Istio, Cosign, and Prometheus.

Phase 5: Controlled Empirical Experiments & Data Collection [Planned — Weeks 8–9]

Execute controlled security attack scenarios and synthetic workload performance tests against both baseline and hardened clusters to collect empirical telemetry.

Phase 6: Comparative Analysis, Evaluation & Documentation [Planned — Weeks 10–12]

Quantify defensive efficacy, latency overhead, and resource trade-offs; synthesize findings into the final project dissertation and academic manuscript.

3. System Development Methodology

To realize the TrustMesh control plane, development is divided into seven modular, sequential implementation stages. Each module is built and validated independently prior to multi-cloud integration:

Stage 1 — Environment Preparation [Planned]

- Configure local container development environments and managed cloud clusters (AWS EKS / Azure AKS).
- Deploy standard sample microservice application (e.g., Google Online Boutique or multi-tier banking demo) to serve as the experimental benchmark testbed.

Stage 2 — Identity & Access Management Layer [Planned]

- Deploy SPIRE server and node agents across cluster nodes for cryptographic workload attestation.
- Establish OIDC identity federation linking Kubernetes Service Accounts to cloud IAM roles (AWS IRSA / Azure Workload Identity), eliminating static long-lived credentials.

Stage 3 — Policy-as-Code Governance Layer [Planned]

- Deploy Open Policy Agent (OPA) / Gatekeeper as a validating admission controller.
- Author declarative Rego security policies enforcing mandatory non-root execution, read-only root filesystems, resource quotas, and namespace isolation.

Stage 4 — Service Mesh & Traffic Security [Planned]

- Install Istio Service Mesh across application namespaces in STRICT mutual TLS (mTLS) mode.
- Configure Layer-7 AuthorizationPolicies restricting service-to-service communication to strictly authorized dependency graphs.

Stage 5 — Continuous Verification & Runtime Integrity [Planned]

- Implement runtime verification and adaptive security response to monitor container hash integrity and detect configuration drift.
- Evaluate adaptive access control policies that respond to detected anomalies (with dynamic trust scoring as an optional enhancement).

Stage 6 — Pipeline & Supply-Chain Security [Planned]

- Harden GitHub Actions CI/CD workflows using secretless OIDC cloud authentication.
- Integrate automated vulnerability scanning (Trivy), SBOM generation (Syft), and container image signing (Cosign/Sigstore) enforced at cluster admission.

Stage 7 — Observability, Auditing & Posture Dashboard [Planned]

- Aggregate authentication events, authorization decisions, and policy violations into Prometheus and Loki.
- Construct a centralized web management dashboard visualizing real-time security posture and audit logs.

4. Threat Model & Planned Experimental Scenarios

To ensure that system evaluation addresses realistic cloud-native vulnerabilities, the project establishes a structured threat model. The eight core threat scenarios, associated business risks, and proposed TrustMesh defensive controls are summarized below:

Threat Scenario	Attack Vector & Technique	Potential Impact	Proposed Control
Unauthorized Workload Access	Attacker or compromised pod invokes internal microservice endpoints without credentials.	Service compromise, data exfiltration	Workload cryptographic identity (SPIFFE) + Layer-7 Istio authorization
Excessive Privileges	Container requests root privileges, hostPath mounts, or cluster-admin API access.	Container breakout, node host compromise	OPA Gatekeeper admission control + least-privilege RBAC policies
Invalid / Spoofed Identity	Attacker presents expired, forged, or un-federated tokens to access service mesh.	Impersonation, unauthorized API access	Strict cryptographic token attestation and OIDC validation
Declarative Policy Violation	Operator or pipeline deploys insecure manifest lacking required security contexts.	Security posture degradation, compliance failure	Automated Policy-as-Code admission blocking + GitOps drift validation
Untrusted Container Image	Attacker injects backdoored image or vulnerable open-source container from public registry.	Supply-chain poisoning, remote code execution	Mandatory Cosign signature verification at cluster admission
Configuration Drift	Out-of-band modification to live Kubernetes deployment bypasses CI/CD static review.	Undetected vulnerability window, policy bypass	Continuous runtime verification daemon + automated alerting
Insecure East-West Traffic	Internal microservice traffic flows unencrypted over shared multi-tenant cluster network.	Packet sniffing, man-in-the-middle, token theft	Strict mutual TLS (mTLS) with automated ephemeral certificate rotation
Compromised CI/CD Secrets	Static cloud IAM keys stored in GitHub repository secrets are leaked or exfiltrated.	Full cloud infrastructure compromise	Secretless ephemeral OIDC cloud authentication for CI/CD runners

Clarification: The above table defines the threat model and planned experimental attack scenarios to be executed in Phase 5. In accordance with project integrity, these attacks have not yet been executed.

5. Data Collection Plan

A cornerstone of this research is that evaluation data will be empirically generated from controlled security experiments deployed on actual cloud-native clusters, rather than relying solely on static public datasets. Telemetry will be systematically collected across six distinct categories:

A. Identity & Authentication Telemetry

- timestamp: ISO-8601 millisecond timestamp of authentication request.
- workload_id: Unique SPIFFE ID (e.g., spiffe://trustmesh.io/ns/prod/sa/order-service).
- auth_method: Authentication protocol (OIDC Token, SPIFFE x509 SVID, Kubernetes Bound Token).
- auth_result: Binary status (SUCCESS / FAILURE / EXPIRED / INVALID_SIGNATURE).
- source_node: Kubernetes node and cloud provider hosting the requesting pod.
- target_resource: Requested service endpoint or cloud API.

B. Authorization & Policy Decision Telemetry

- requesting_identity: Verified identity of calling workload.
- target_service: Destination service and URI path.
- requested_action: HTTP verb or API operation (GET, POST, DELETE, kubectl apply).
- policy_id: Specific OPA Rego rule or Istio AuthorizationPolicy identifier.
- policy_decision: Enforcement outcome (ALLOW / DENY / QUARANTINE).
- evaluation_latency_ms: Milliseconds taken by OPA engine to evaluate request.
- violation_reason: Diagnostic reason code for denied operations.

C. Service Mesh Network Traffic Data

- source_service & dest_service: Service names communicating across the mesh.
- mtls_status: Cryptographic channel state (STRICT_MUTUAL_TLS / PLAINTEXT / FAILED_HANDSHAKE).
- cipher_suite: Negotiated TLS cipher suite (e.g., TLS_AES_128_GCM_SHA256).
- request_status_code: HTTP response status (200, 401, 403, 500).
- network_latency_ms: End-to-end request round-trip time.
- connection_reused: Boolean indicating whether HTTP/2 connection pooling was leveraged.

D. Security Violation & Event Data

- experiment_scenario: ID of active experimental attack scenario.
- event_type: Category of detected breach (UNAUTHORIZED_EGRESS, ROOT_CONTAINER_ATTEMPT, DRIFT_DETECTED).
- severity: Severity level (LOW / MEDIUM / HIGH / CRITICAL).
- detection_latency_ms: Milliseconds elapsed between attack injection and alert generation.
- enforcement_action: Immediate automated mitigation applied (BLOCKED_AT_ADMISSION / POD_EVICTED / TRAFFIC_DROPPED).

E. System Performance & Overhead Telemetry

- request_latency_p50_p95_p99: Millisecond latency percentiles under load.
- throughput_rps: Successful requests per second sustained by services.
- cpu_utilization_percent: CPU cores consumed by application vs. sidecar proxies.
- memory_footprint_mb: Resident set size (RSS) memory consumed by Envoy proxies and OPA agents.
- error_rate_percentage: Ratio of failed requests under stress.

F. Software Supply Chain Verification Telemetry

- artifact_digest: SHA-256 cryptographic digest of container image.
- signature_status: Cosign digital signature verification state (VALID / UNTRUSTED / MISSING).
- sbom_package_count: Number of software dependencies cataloged in Syft SBOM.
- critical_cve_count: Number of high/critical vulnerabilities identified by Trivy scanner.
- admission_result: Cluster admission outcome (DEPLOYED / REJECTED_BY_POLICY).

6. Planned Experimental Data Schema

During Phase 5 experimental trials, telemetry logs will be normalized and exported into a unified tabular structure for statistical analysis. The formal schema design is presented below:

Attribute Name	Data Type	Example Format	Description
timestamp	STRING	2026-10-15T14:22:05.120Z	ISO-8601 execution timestamp
scenario_id	STRING	EXP-04-POLICY-VIOLATION	Experimental test case identifier
source_identity	STRING	spiffe://trustmesh.io/ns/dev/sa/frontend	Cryptographic SPIFFE workload identity
target_service	STRING	backend-orders-api	Destination service name
requested_action	STRING	POST /v1/checkout	API operation or HTTP method
auth_result	ENUM	SUCCESS | FAILED | EXPIRED	Identity verification outcome
policy_result	ENUM	ALLOW | DENY | QUARANTINE	OPA / Gatekeeper admission decision
mtls_status	ENUM	MUTUAL_TLS | PLAINTEXT	Network encryption channel state
security_event	STRING	PRIVILEGE_ESCALATION_BLOCKED	Triggered security detection label
latency_ms	FLOAT	4.25	End-to-end request latency in milliseconds
cpu_usage_mcores	INTEGER	48	Pod/sidecar CPU consumption in millicores
memory_usage_mb	FLOAT	78.4	Pod/sidecar memory footprint in megabytes
throughput_rps	FLOAT	1250.0	Active system throughput in requests per second
signature_valid	BOOLEAN	TRUE | FALSE	Cosign image signature verification state

Note on Academic Integrity: This schema outlines the planned data collection fields. No synthetic or invented measurements have been inserted; values will be populated during experimental trials.

7. Experimental Scenarios Matrix

To empirically validate security controls, eight controlled test scenarios will be executed against the testbed cluster during Phase 5. The test matrix and expected system behavior are defined below:

Test ID	Scenario Description	Experimental Action	Expected Result	Verification Method
EXP-01	Authorized Workload Communication	Pod with valid SPIFFE SVID calls authorized microservice endpoint.	**Allow**	Request succeeds with HTTP 200 over strict mTLS; latency recorded.
EXP-02	Unauthenticated Workload Access	Pod lacking SPIFFE x509 certificate attempts to invoke internal service.	**Deny**	mTLS handshake fails immediately at Envoy ingress proxy; request dropped.
EXP-03	Unauthorized Service-to-Service Call	Authenticated service attempts to call unapproved backend (e.g., frontend calling payment DB).	**Deny**	Layer-7 Istio AuthorizationPolicy rejects request with HTTP 403 Forbidden.
EXP-04	Privilege Escalation Manifest Admission	Deployment manifest requests privileged container securityContext or hostPID.	**Block / Reject**	OPA Gatekeeper admission webhook rejects manifest; deployment fails with error.
EXP-05	Untrusted Container Image Deployment	Deployment references image not cryptographically signed by authorized Cosign key.	**Block / Reject**	Cosign admission controller blocks pod scheduling; alert logged.
EXP-06	Out-of-Band Configuration Drift	Attacker modifies live Kubernetes deployment to expose an unauthorized port.	**Detect & Alert**	Detection latency will be measured during the experiment.
EXP-07	Vulnerable Dependency Microservice	Microservice with critical CVSS >= 9.0 CVE is admitted for testing.	**Adaptive Response**	Runtime verification flags vulnerability; adaptive policy restricts database access.
EXP-08	Synthetic Load Stress Test	Controlled synthetic load at multiple request rates generated across mesh to measure overhead.	**Process & Measure**	Latency percentiles (p50, p95, p99), CPU, and memory recorded.

8. Baseline vs. Zero-Trust Comparative Evaluation Methodology

To scientifically demonstrate the defensive efficacy and compute trade-offs of the TrustMesh framework, the project will conduct a rigorous comparative study between two identical testbed environments:

Environment A: Baseline Cloud-Native Setup (Without Zero-Trust Controls)

- **Networking:** Standard Kubernetes CNI (flat network); all pods can communicate with all other pods across namespaces without encryption.
- **Identity:** Default Kubernetes Service Accounts with static cluster role bindings; long-lived static API credentials.
- **Policy:** Standard native Kubernetes RBAC; no validating admission controller or Policy-as-Code.
- **Supply Chain:** Standard public image registry pulls without mandatory digital signature verification.

Environment B: Hardened ZeroTrustCloud Setup (With TrustMesh Controls)

- **Networking:** Istio Service Mesh with STRICT mutual TLS, ephemeral certificate rotation, and default-deny NetworkPolicies.
- **Identity:** SPIRE cryptographic workload attestation with ephemeral x509 SVIDs and OIDC cloud federation.
- **Policy:** OPA Gatekeeper validating admission controller enforcing declarative Rego policies.
- **Supply Chain:** Mandatory Cosign cryptographic signature verification and Trivy vulnerability gatekeeping.

Comparative Evaluation Dimensions:

1. Security Efficacy: Percentage of unauthorized lateral movement attempts, privilege escalation requests, and unsigned container deployments successfully blocked.

2. Latency Overhead: Delta in p50, p95, and p99 request latency introduced by Envoy mTLS cryptographic handshakes and OPA policy evaluation.

3. Throughput Impact: Maximum sustained requests per second (RPS) achievable before saturating CPU/RAM.

4. Resource Footprint: Additional CPU millicores and RAM megabytes consumed by sidecar proxies and control plane daemons.

9. Formal Evaluation Metrics

The evaluation will quantitatively measure system performance across seven core categories:

Evaluation Category	Evaluation Metric	Measurement Formula / Method
Security Enforcement	Unauthorized Access Prevention Rate (%)	Blocked unauthorized access requests / Total unauthorized attempts * 100
Identity Verification	Token Attestation Latency (ms)	Time taken to cryptographically issue and verify SPIFFE SVIDs
Policy Governance	Policy Evaluation Latency (ms)	Time taken by OPA Gatekeeper to evaluate admission manifests
Mesh Security	mTLS Encryption Overhead (ms)	Latency delta between plaintext HTTP and strict mTLS communication
Supply Chain	Artifact Verification Time (ms)	Time taken by Cosign admission webhook to verify container signatures
Performance Cost	Throughput Degradation Rate (%)	((Baseline RPS - ZeroTrust RPS) / Baseline RPS) * 100
Resource Bloat	Memory & CPU Overhead per Pod	Resident Set Size (MB) and CPU cores allocated to Envoy sidecars
Observability	Audit Telemetry Coverage (%)	Proportion of security events successfully captured in central logs

10. Proposed Zero-Trust Architecture Blueprint

The proposed TrustMesh security framework is structured as a multi-tier control plane orchestrating identity, declarative policy, mesh encryption, and continuous runtime verification:

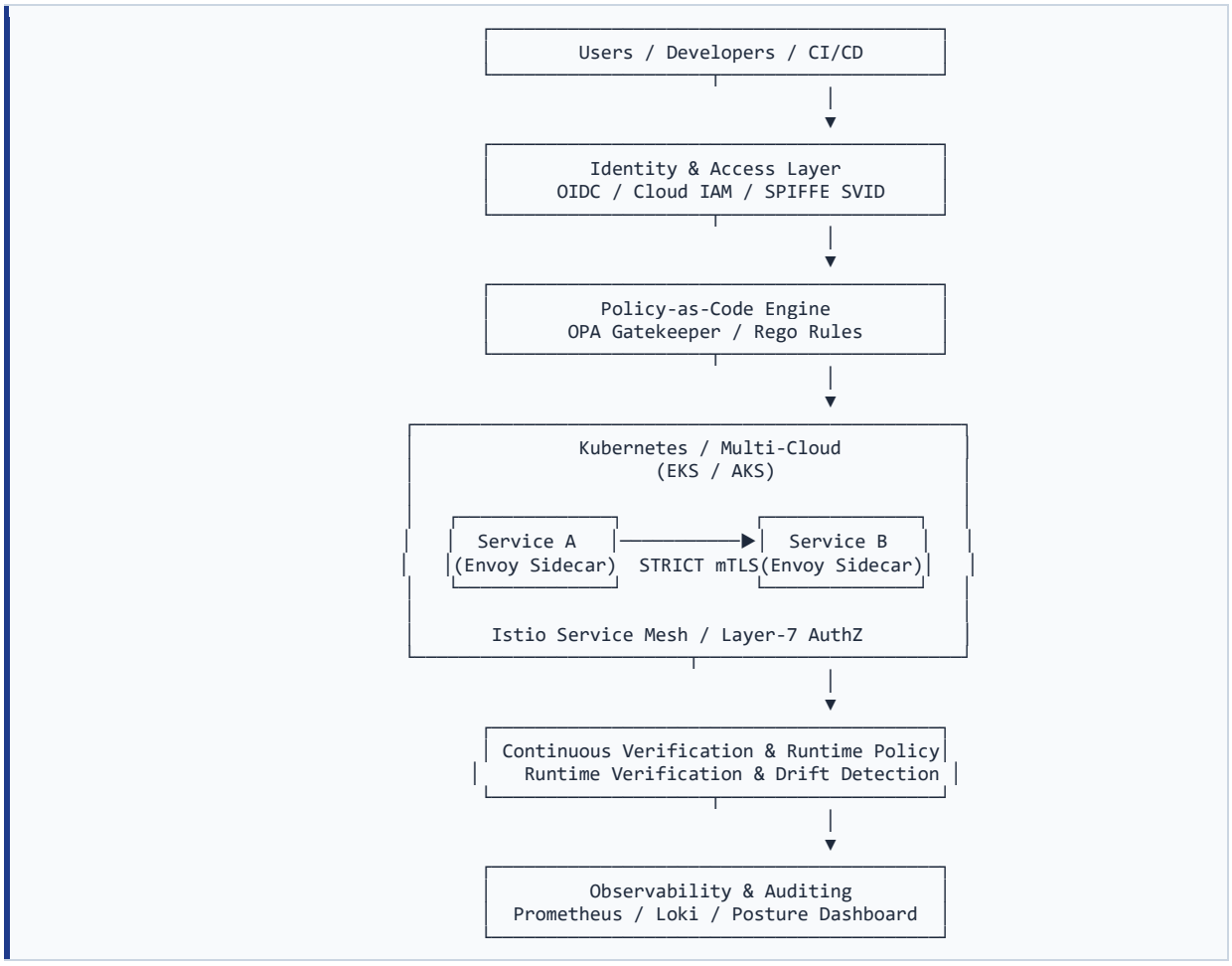


Figure 1: Architectural Blueprint of the proposed TrustMesh control plane, illustrating hierarchical request flows from identity inception through policy gating, mesh encryption, and continuous runtime audit.

11. GitHub Repository Blueprint: Current vs. Planned Structure

To maintain strict transparent tracking throughout our final-year project, the GitHub repository is organized to clearly distinguish assets that exist currently from modules planned for future development phases:

Current Repository State (Week 4 — Existing Files)

```
ZeroTrustCloud-A-Zero-Trust-Architecture-Framework/
│
├── README.md                                          [Current — High-level project summary]
│
├── Plannings/                                         [Current — Project work plans & methodology]
│   ├── Work_Plan_and_Project_Progress_Report_ZeroTrustCloud.pdf
│   └── ZeroTrustCloud_Methodology_Data_Collection_Experimental_Plan.pdf
│
└── Research Papers Review/                            [Current — Literature survey reports]
    ├── part 1/                                        [Current — Phase I literature reviews]
    │   ├── Baire Gowda_LiteratureReview_ZeroTrustCloud.pdf
    │   ├── Hithaishi-sp-Research_Review_ZeroTrustCloud_TrustMesh.pdf
    │   └── Sanjay REVIEW OF RESEARCH PAPER-1.pdf
    │
    └── part 2/                                        [Current — Phase II literature reviews]
        ├── Baire_Gowda_LiteratureReview_ZeroTrustCloud_Week3.pdf
        ├── Hithaishi-sp-Research_Review_ZeroTrustCloud_TrustMesh.pdf
        └── SANJAY LITERATURE REVIEW 2.pdf
```

Planned Future Repository Structure (To be Created During Implementation)

```
ZeroTrustCloud-A-Zero-Trust-Architecture-Framework/ (Future Extension)
│
├── identity-engine/          [Planned: Phase 4 — SPIRE attestation & OIDC federation manifests]
├── policy-engine/            [Planned: Phase 4 — OPA Gatekeeper constraints & Rego policies]
├── service-mesh/             [Planned: Phase 4 — Istio STRICT mTLS & Layer-7 AuthZ policies]
├── continuous-verification/  [Planned: Phase 5 — Runtime verification daemon & drift detectors]
├── supply-chain-security/    [Planned: Phase 5 — Cosign signature webhooks & SBOM generators]
├── observability/            [Planned: Phase 6 — Prometheus telemetry configs & Grafana dashboards]
│
├── infrastructure/           [Planned: Phase 3 — Cluster provisioning automation]
│   ├── kubernetes/           [Planned — Base Helm charts & microservice testbed]
│   ├── aws/                  [Planned — AWS EKS Terraform & IRSA IAM roles]
│   └── azure/                [Planned — Azure AKS Terraform & Workload Identity]
│
└── experiments/              [Planned: Phase 5 — Empirical evaluation]
    ├── scenarios/            [Planned — Attack simulation scripts EXP-01 to EXP-08]
    ├── datasets/             [Planned — Raw collected CSV telemetry]
    └── results/              [Planned — Comparative performance charts & logs]
```

Clarification: The planned folders above represent the future modular directory structure that will be incrementally populated as code is authored during Weeks 5–10.

12. Week 4 Progress Connection & Next Phase Roadmap

Current Week 4 Accomplishments

- **Methodology Formalization:** Completed rigorous research and development methodology mapping NIST SP 800-207 principles to Kubernetes.
- **Threat Model Structuring:** Constructed formal 8-scenario cloud-native threat model detailing specific attack vectors and defensive controls.
- **Data Collection Engineering:** Defined 6 categories of empirical telemetry data and formulated a normalized 14-attribute experimental CSV schema.
- **Experimental Test Matrix:** Specified 8 controlled security scenarios (EXP-01 through EXP-08) comparing baseline vs. Zero-Trust postures.
- **Evaluation Metrics Definition:** Formulated quantitative mathematical formulas for measuring defensive efficacy, latency delta, and resource overhead.
- **Repository Governance:** Established clean GitHub repository blueprint strictly separating current research assets from planned implementation folders.

Next Phase Roadmap (Weeks 5–7)

1. **Environment Provisioning (Week 5):** Establish local and cloud development Kubernetes clusters; deploy sample microservice testbed.
2. **Baseline Characterization (Week 5):** Execute synthetic traffic load across unhardened baseline cluster to record initial latency and throughput benchmarks.
3. **Zero-Trust Control Implementation (Weeks 6–7):** Deploy SPIRE workload attestation, configure OPA Gatekeeper admission rules, and enforce Istio STRICT mTLS.
4. **Empirical Testing & Data Collection (Weeks 8–9):** Execute attack simulations EXP-01 to EXP-08; record empirical telemetry into normalized CSV datasets.
5. **Evaluation & Dissertation Finalization (Weeks 10–12):** Analyze comparative data, generate performance graphs, and author final project thesis.